PRACTICAL 2-A

Object Oriented Programming

AIM:

To implement and understand the concepts specified in Practical 2 of the CEUE203 Object Oriented Programming practical list, using Java. The work covers Part A: Thermostat, Cinema Show, Parking Lot and Part B: Customer and Account encapsulation.

CODE SCREENSHOT:

CinemaShow.java

Part A > Practical 2-A > CinemaShow.java > ...

```java
public class CinemaShow {
    private String title;
    private int seatsAvailable;
    private final int capacity;
    private static int totalBooked = 0;

    public CinemaShow(String title, int capacity) {
        this.title = title;
        this.capacity = capacity;
        seatsAvailable = capacity;
    }

    public CinemaShow(String title) {
        this(title, capacity: 100);
    }

    public boolean book(int n) {
        if (n > 0 && n <= seatsAvailable) {
            seatsAvailable -= n;
            totalBooked += n;
            return true;
        }
        return false;
    }

    public void cancel(int n) {
        if (n > 0)
            seatsAvailable = Math.min(capacity, seatsAvailable + n);
    }

    public int getSeatsAvailable() {
        return seatsAvailable;
    }

    public static int getTotalBooked() {
        return totalBooked;
    }

    Run | Debug
    public static void main(String[] args) {
        CinemaShow show = new CinemaShow(title: "Java Premiere", capacity: 50);
        System.out.println("Book 20: " + show.book(n: 20) + ", seats: " + show.getSeatsAvailable());
        System.out.println("Book 40: " + show.book(n: 40) + ", seats: " + show.getSeatsAvailable());
        show.cancel(n: 5);
        System.out.println("After cancellation, seats: " + show.getSeatsAvailable());
        System.out.println("Total booked: " + getTotalBooked());
    }
}
```

Code Screenshot 1

ParkingLot.java

Part A > Practical 2-A > ParkingLot.java > ...

```java
public class ParkingLot {
    private int twoWheelers = 0, fourWheelers = 0;
    private final int twoCap, fourCap;
    private static long revenue = 0;

    public ParkingLot(int twoCap, int fourCap) {
        this.twoCap = twoCap;
        this.fourCap = fourCap;
    }

    public boolean park(String type) {
        if (type.equalsIgnoreCase("two") && twoWheelers < twoCap) {
            twoWheelers++;
            revenue += 20;
            return true;
        }
        if (type.equalsIgnoreCase("four") && fourWheelers < fourCap) {
            fourWheelers++;
            revenue += 40;
            return true;
        }
        System.out.println("Full");
        return false;
    }

    public void leave(String type) {
        if (type.equalsIgnoreCase("two") && twoWheelers > 0)
            twoWheelers--;
        else if (type.equalsIgnoreCase("four") && fourWheelers > 0)
            fourWheelers--;
    }

    Run | Debug
    public static void main(String[] args) {
        ParkingLot p = new ParkingLot(twoCap: 2, fourCap: 1);
        p.park(type: "two");
        p.park(type: "four");
        p.park(type: "two");
        p.park(type: "two");
        p.leave(type: "two");
        System.out.println(
                "Two-wheelers: " + p.twoWheelers + ", Four-wheelers: " + p.fourWheelers + ", Revenue: " + revenue);
    }
}

```

Code Screenshot 2

Thermostat.java 1

Part A > Practical 2-A > Thermostat.java > ...

```java
public class Thermostat {
    private String location;
    private int temperature;
    private static final int MIN = 16, MAX = 30;
    private static int activeCount = 0;

    public Thermostat(String location, int startTemp) {
        this.location = location;
        temperature = (startTemp >= MIN && startTemp <= MAX) ? startTemp : 22;
        activeCount++;
    }

    public Thermostat(String location) {
        this(location, startTemp: 22);
    }

    public void raise() {
        if (temperature < MAX)
            temperature++;
        else
            System.out.println("Already at maximum (30)");
    }

    public void lower() {
        if (temperature > MIN)
            temperature--;
        else
            System.out.println("Already at minimum (16)");
    }

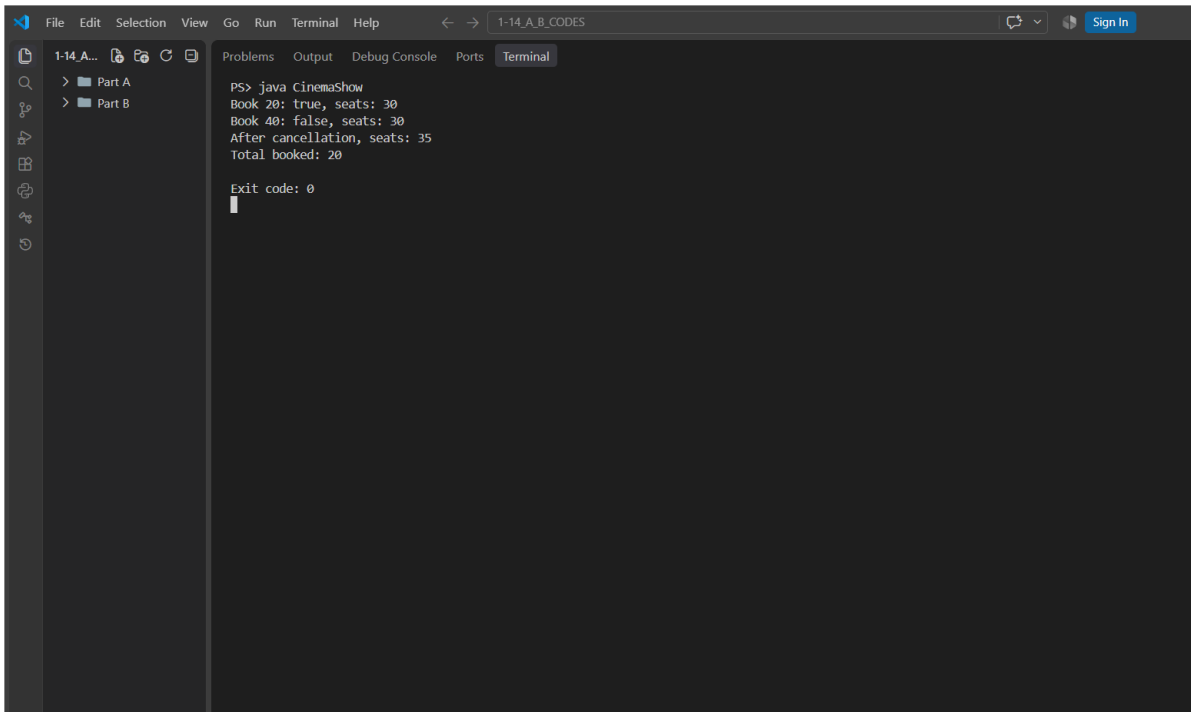
    public int getTemperature() {
        return temperature;
    }

    public static int getActiveCount() {
        return activeCount;
    }

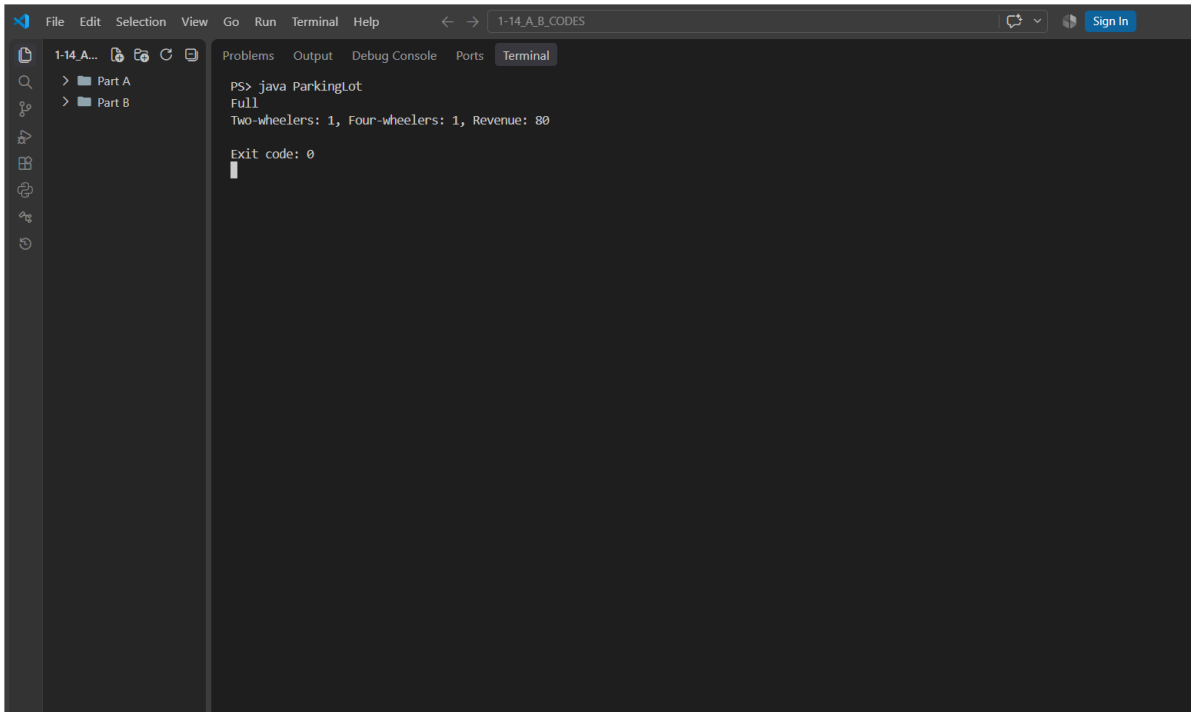
    Run | Debug
    public static void main(String[] args) {
        Thermostat a = new Thermostat(location: "Lab", startTemp: 20), b = new Thermostat(location: "Office");
        for (int i = 0; i < 10; i++) {
            a.raise();
            System.out.println(a.location + ": " + a.getTemperature());
        }
        for (int i = 0; i < 20; i++) {
            a.lower();
            System.out.println(a.location + ": " + a.getTemperature());
```

Code Screenshot 3

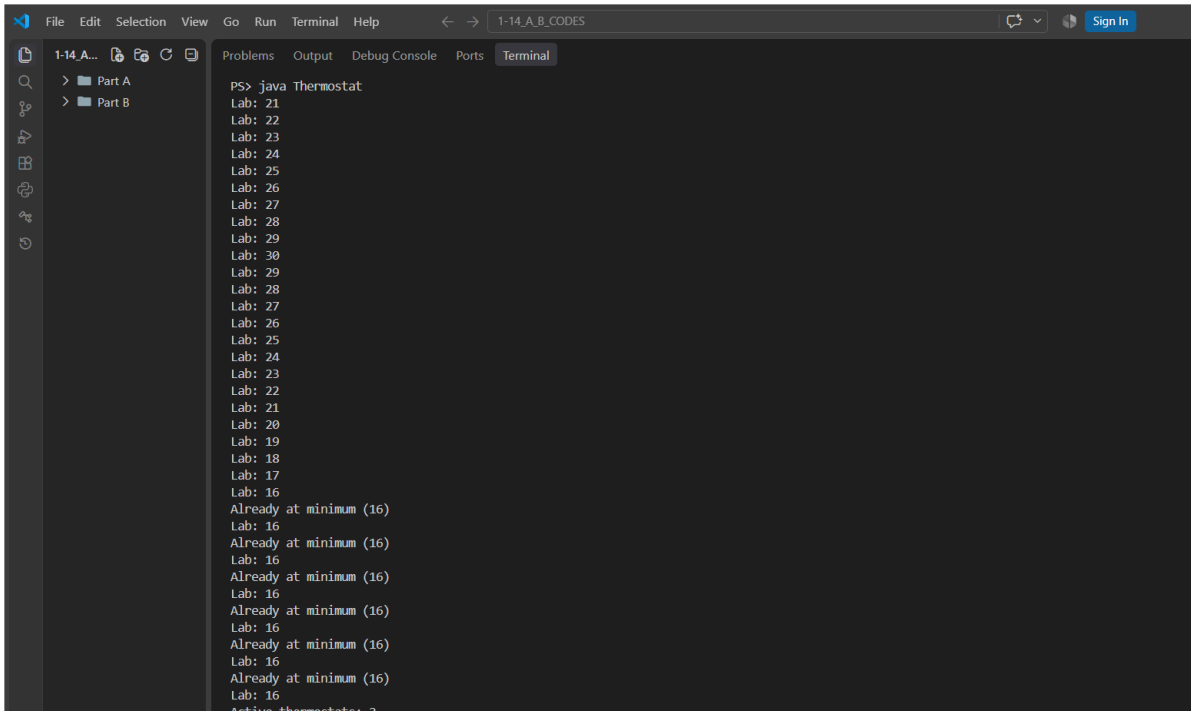
OUTPUT SCREENSHOT:



Output Screenshot 1



Output Screenshot 2



Output Screenshot 3

LEARNING OUTCOMES:

Apply classes, constructors, encapsulation, static members, and object arrays to model real-world entities.